SIS 3204Y(5)
Natural Language Processing

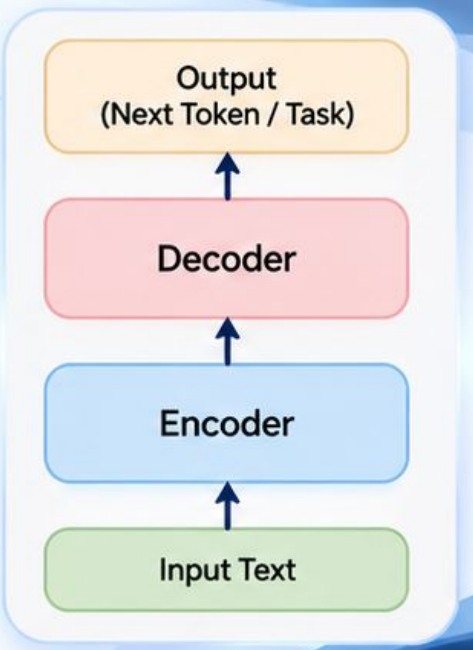
Week 7

Transformer Architecture, BERT and GPT

From Attention to Powerful Language Models

Transformer Architecture | Encoder–Decoder Models | BERT | GPT









Learning Outcomes

By the end of this lecture, you should be able to:

Describe the Transformer architecture

Explain the main components of encoder and decoder layers, and how multi-head self-attention, feed-forward networks, residual connections and layer normalisation work together.

Explain encoder–decoder models

Describe how encoder–decoder Transformers process input and generate output, and understand the roles of masked self-attention and cross-attention in the decoder.

Understand BERT

Explain the architecture and pretraining objectives of BERT, and how it is applied to downstream NLP tasks such as text classification, named entity recognition and question answering.

Understand GPT

Explain the decoder-only architecture and autoregressive next-token prediction, and how GPT is trained and used for text generation and other NLP applications.

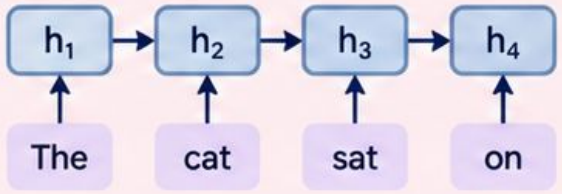
Compare and select appropriate Transformer models

Compare encoder-only, decoder-only and encoder–decoder models, and justify the choice of an appropriate model for a given NLP application.



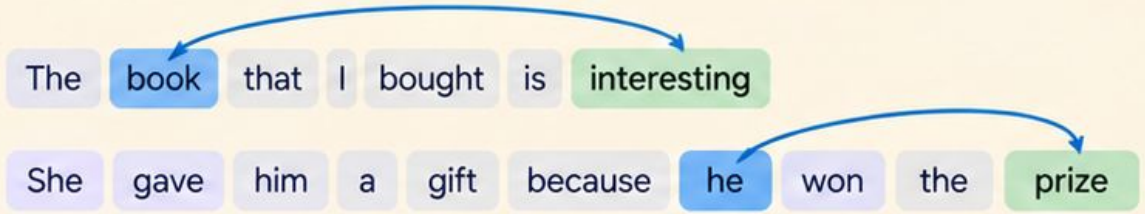
Why Transformers Changed NLP

Limitations of traditional sequential models



- Process tokens one at a time
- Struggle to capture long-range dependencies
- Information can be lost over long sequences
- Training is slower due to sequential computation

Examples of long-range dependencies



Need for a new architecture



- Handle long-range dependencies more effectively
- Process tokens in parallel
- Be more efficient and scalable
- Learn richer and more contextual representations
- Support a wide range of NLP tasks

Advantages of the Transformer architecture



Captures long-range context effectively
Each token can directly attend to all other tokens in the sequence.



Parallel processing
All tokens are processed simultaneously, leading to faster training.



More scalable
Handles larger datasets and longer sequences more efficiently.



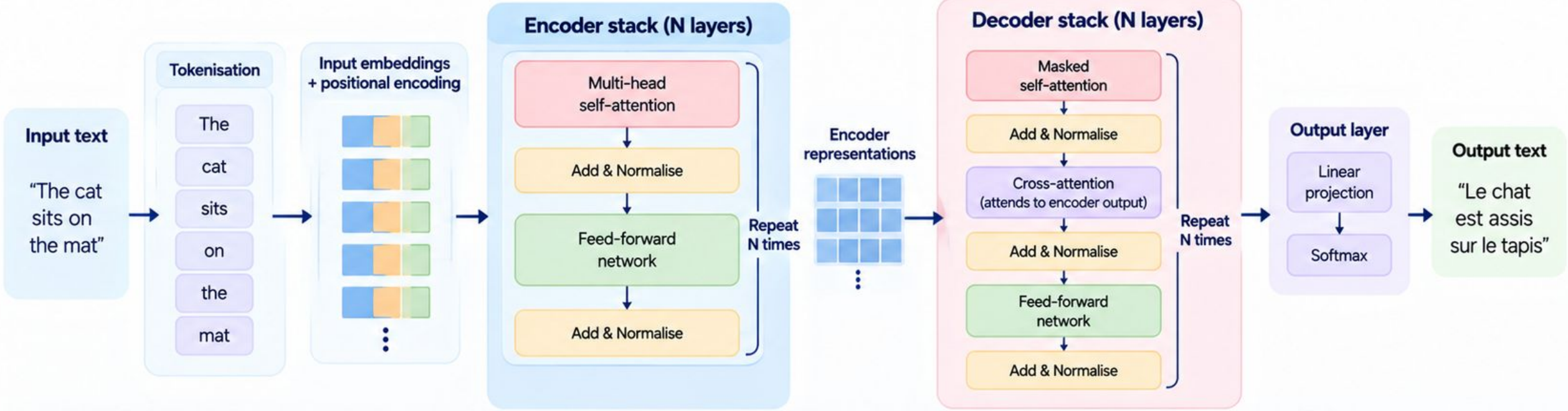
Richer contextual representations
Self-attention learns context-dependent representations for each token.



Versatile and high performance
Forms the foundation of modern state-of-the-art NLP models such as BERT, GPT and many others.

Transformer Architecture – Big Picture

The original Transformer consists of an encoder stack and a decoder stack connected through attention.



Encoder

- Processes the input sequence using self-attention.
- Produces contextual representations for each token.

Decoder

- Generates the output sequence autoregressively.
- Uses masked self-attention and cross-attention to attend to encoder representations.

Attention connection

- The decoder attends to the encoder outputs through cross-attention.
- Allows the model to use relevant information from the input sequence when generating each token.

Output

- A linear layer maps decoder representations to a vocabulary of tokens.
- Softmax converts the scores to probabilities for the next token.

Positional Encoding

Transformers do not have a built-in notion of word order, so positional information is added to the token embeddings.

Why positional encodiing is needed

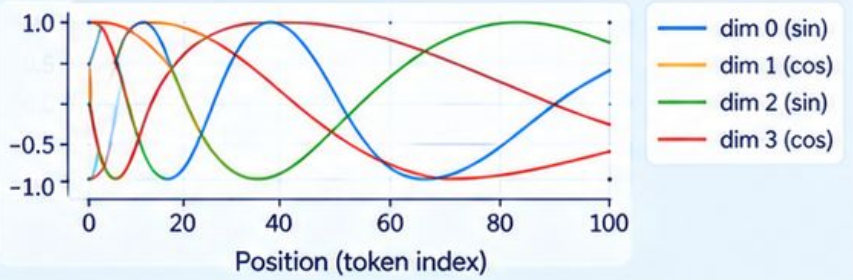
- Self-attention treats the input tokens as a set of vectors.
- Without positional information, the model would not know the order of the tokens.
- Positional encoding provides information about the position of each token in the sequence.

Sinusoidal positional encoding (original Transformer)

Uses sine and cosine functions of different frequencies.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d\ mosel}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d\ mosel}}\right)$$



Learned positional encoding (used in many modern models)

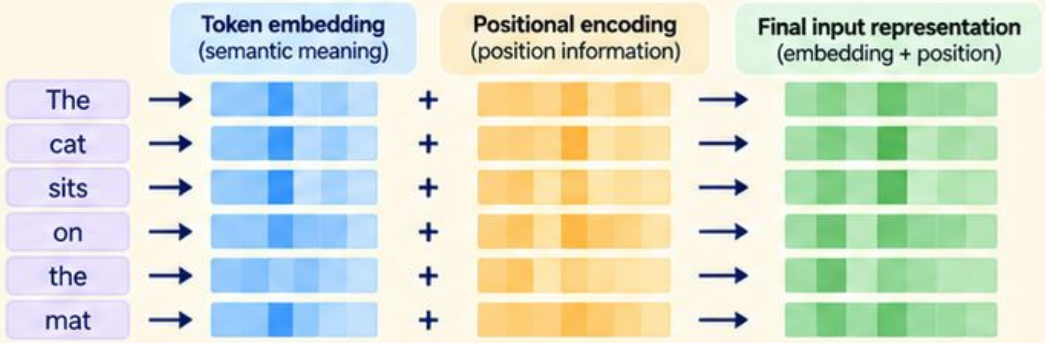
Each position has a learned embedding vector, trained together with the model.

	Dim 1	Dim 2	Dim 3	Dim 4	...	Dim d_{model}
Position 0	0.12	-0.34	0.56	0.21	...	-0.18
Position 1	0.03	0.41	-0.22	0.67	...	0.11
Position 2	-0.28	0.19	0.33	-0.14	...	0.27
Position 3	0.45	-0.07	-0.11	0.38	...	-0.09
⋮	⋮	⋮	⋮	⋮		⋮

- Learned during training
- Can adapt to the data and model architecture
- Widely used in modern transformer models (e.g. BERT, GPT)

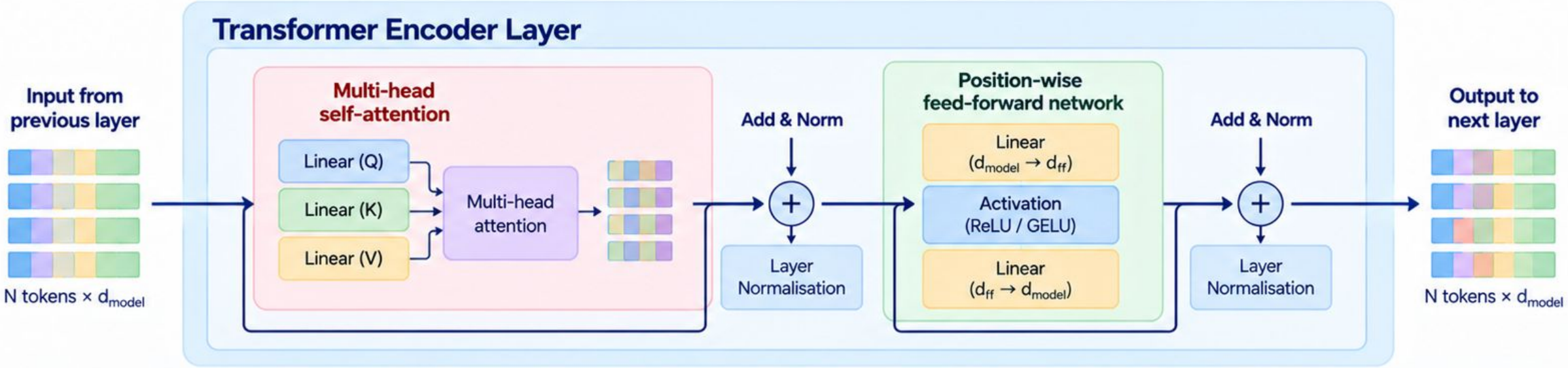
Input embedding with positional encoding

The final input representation is the sum of the token embedding and the positional encoding.



Inside a Transformer Encoder Layer

Each encoder layer processes the input sequence using self-attention and a feed-forward network, with residual connections and layer normalisation.



1 Multi-head self-attention

- Each token attends to all other tokens in the input sequence.
- Multiple attention heads capture different relationships in parallel.
- Produces contextualised representations for each token.

2 Add & Norm (after attention)

- Residual connection adds the original input to the attention output.
- Layer normalisation stabilises training and helps information flow across layers.

3 Position-wise feed-forward network

- A fully connected neural network applied to each token independently.
- Typically consists of two linear transformations with a non-linear activation (e.g. ReLU or GELU).
- Increases the model's capacity to learn complex representations.

4 Add & Norm (after feed-forward)

- Another residual connection adds the attention output to the feed-forward output.
- Layer normalisation is applied again to produce the final output of the encoder layer.

SIS 3204Y(5)
Natural Language Processing

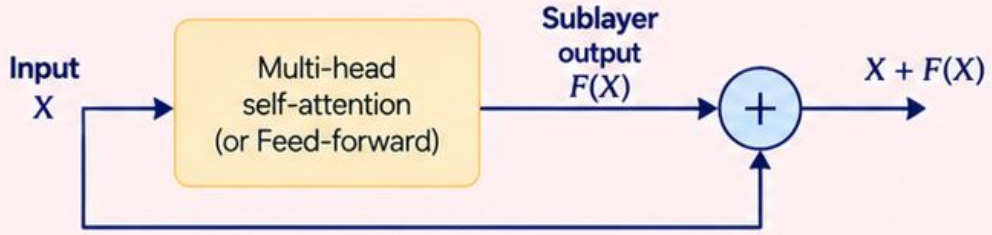
Week 7
Transformer Architecture, BERT and GPT

Residual Connections and Layer Normalisation

Residual connections and layer normalisation help with information flow, stabilise training and improve model performance.

Residual connections

The input to a sub-layer is added to the sub-layer output (skip connection).



- Helps to preserve the original input information.
- Allows gradients to flow more easily through deep networks (reduces vanishing gradient problem).
- Enables the model to learn smaller updates (the sub-layer can focus on learning a refinement instead of a complete transformation).
- Improves training stability and allows for deeper models.

Layer normalisation

Normalises the activations for each token across its feature dimensions.

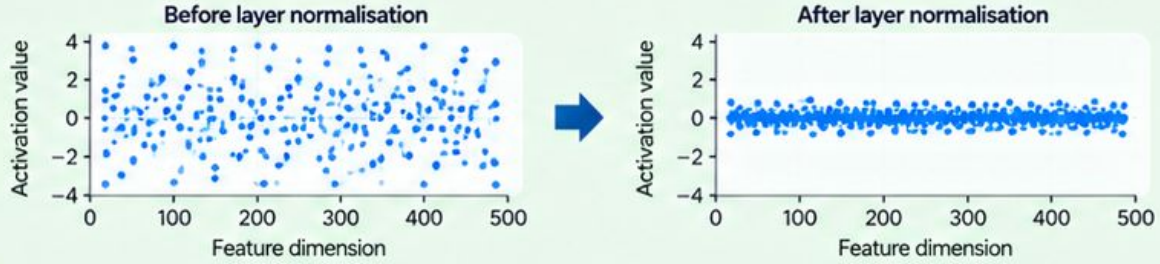
Input for one token

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_d \end{bmatrix} \rightarrow \text{Layer Normalisation} \rightarrow y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_d \end{bmatrix}$$

What it does (per token)

- Computes the mean and variance of the token's features.
- Normalises the features to have zero mean and unit variance.
- Learns scale (γ) and shift (β) parameters.
- Applied after the residual addition.

Effect on activations



Key benefits

- More stable and faster training
- Improved gradient flow in deep models
- Better performance and generalisation
- Widely used in all modern Transformer models

Where it is applied

- After the residual addition in each sub-layer (self-attention and feed-forward).
- Used in both encoder and decoder layers.
- Part of the standard Transformer architecture.

Variation

- Some modern models use Pre-Layer Normalisation (applying normalisation before the sub-layer) instead of the original Post-Layer Normalisation.

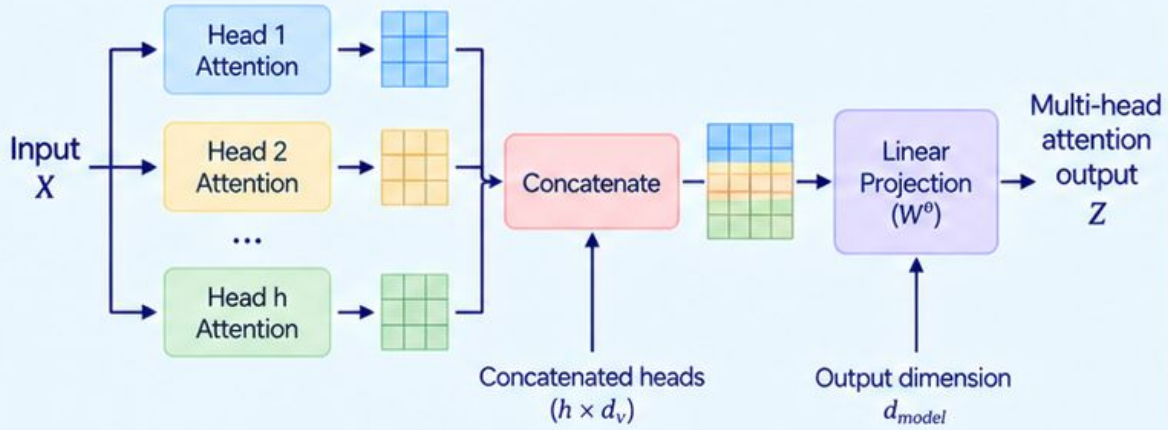
Multi-Head Attention

Multi-head attention allows the model to attend to information from different representation subspaces simultaneously.

Why multiple heads?

- A single attention head can only focus on one type of relationship at a time.
- Multi-head attention uses multiple attention heads in parallel to capture different relationships in the data.
- Each head can learn to focus on different aspects (e.g. syntax, semantics, position).
- The outputs are then combined to produce a richer representation.

Multi-head attention architecture



Mathematical formulation

For h attention heads:

1. **Compute attention for each head:**

$$head_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$
$$\text{for } i = 1, 2, \ldots, h$$

2. **Concatenate the heads:**

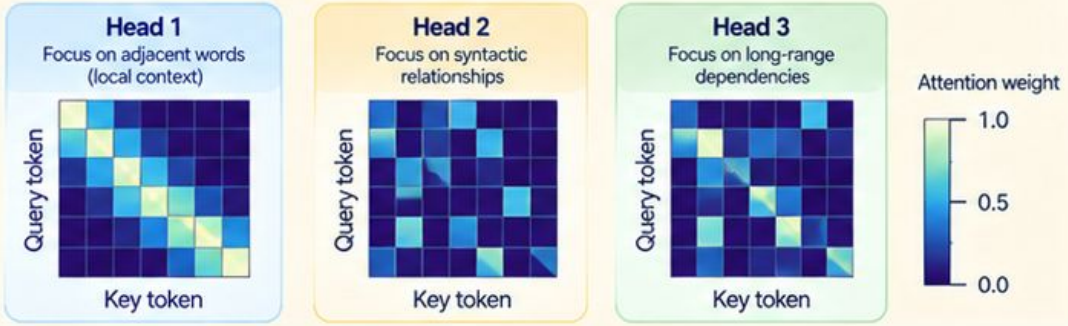
$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W^O$$

3. **Parameter shapes:**

- $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{model} \times d_k}$
- $W^O \in \mathbb{R}^{hd_v \times d_{model}}$
- h: number of attention heads
- $d_k = d_v = d_{model} / h$ (typically)

Example: different attention heads

Each head can learn to focus on different relationships.



Key benefits

- Allows the model to attend to multiple types of information simultaneously.
- Captures both local and long-range dependencies.
- Learns different relationships in parallel (e.g. syntactic, semantic, positional).
- Improves the model's ability to understand complex language structures.
- Widely used in all modern Transformer-based models.

Typical configuration

Common settings in practice:

Model	d_{model}	Number of heads (h)	$d_k = d_v$
BERT (base)	768	12	64
GPT (small)	768	12	64
GPT-3 (large)	12288	96	128
Llama 3	4096	32	128

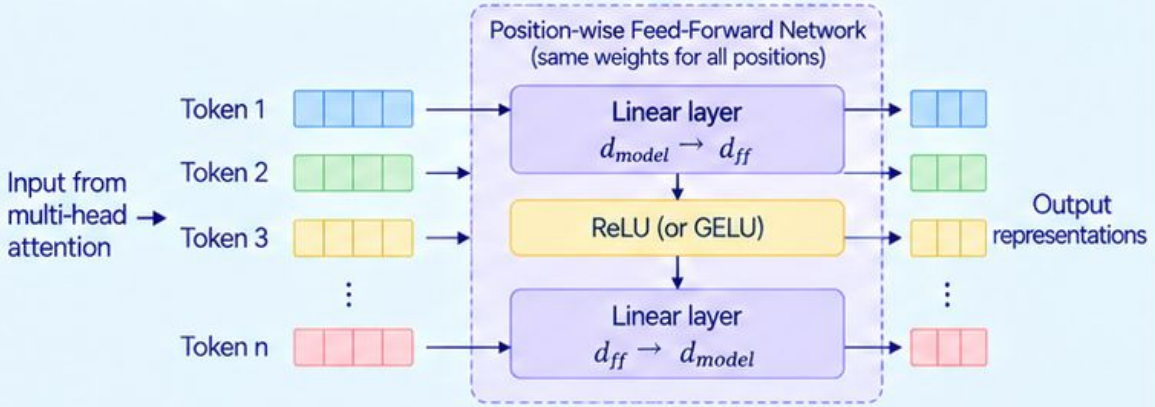
Position-Wise Feed-Forward Networks (FFN)

The position-wise feed-forward network (FFN) is a simple fully connected neural network applied independently to each position in the sequence.

Key idea

- Applied independently and identically to each position (token) in the sequence.
- Consists of two linear transformations with a hon-linear activation function between them.
- Introduces non-linearity and increases the model's representational power.
- The same weights are shared across all positions.

Position-wise FFN in the Transformer encoder/decoder



Mathematical formulation

For each position i in the sequence:

$$\mathrm{FFN}(x_i) = W_2\ \sigma(W_1 x_i + b_1) + b_2$$

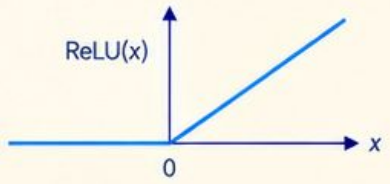
Where:

- $x_i \in \mathbb{R}^{d_{model}}$: input representation at position i
- $W_1 \in \mathbb{R}^{d_{ff} \times d_{model}}$: first linear layer
- $b_1 \in \mathbb{R}^{d_{ff}}$: first layer bias
- $\sigma(\cdot)$: non-linear activation (ReLU or GELU)
- $W_2 \in \mathbb{R}^{d_{model} \times d_{ff}}$: second linear layer
- $b_2 \in \mathbb{R}^{d_{model}}$: second layer bias
- d_{model} : model dimension
- d_{ff} : feed-forward dimension (typically $4 \times d_{model}$)

Activation functions

ReLU (Rectified Linear Unit)

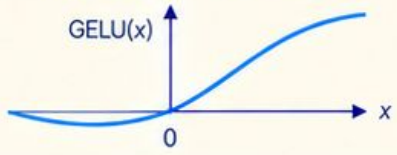
$$\mathrm{ReLU}(x) = \max(0, x)$$



GELU (Gaussian Error Linear Unit)

$$\mathrm{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the standard normal cumulative distribution.



Typical dimensions

Model	d_{model}	d_{ff}	Ratio (d_{ff} / d_{model})
BERT (base)	768	3072	4
BERT (large)	1024	4096	4
GPT-3 (small)	768	3072	4
GPT-3 (large)	12288	49152	4
Llama 3	4096	11008	≈ 2.7

Typically, $d_{ff} = 4 \times d_{model}$ (but can vary by model).

Key points

- Applied to each position independently (same weights for all positions).
- Two linear layers with a non-linear activation (ReLU or GELU).
- Increases the model's capacity to learn complex patterns.
- Used in both encoder and decoder stacks.
- Together with multi-head attention, it forms the core computation in each Transformer layer.

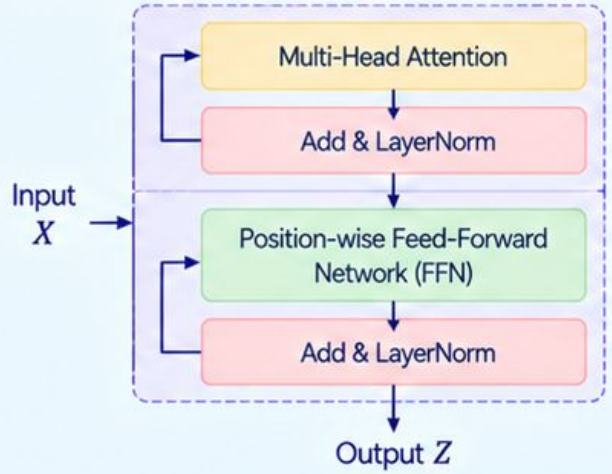
Layer Normalsation (LayerNorm)

Layer Normalisation (LayerNorm) normalises the activations across the features of each token, improving training stability.

Why normalisation is needed?

- Activations can have different scales during training.
- This can slow down training and make it unstable.
- Layer normalisation normalises the activations for each token independently.
- It helps the model train faster, be more stable, and work well with deep architectures.

Where is LayerNorm used in the Transformer?



LayerNorm is applied after each sublayer (attention and FFN) together with a residual connection (Add & LayerNorm).

Mathematical formulation

For an input vector $x \in \mathbb{R}^d$ (e.g. the representation of a token):

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Where:

- $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$ (mean of the features)
- $\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$ (variance of the features)
- $\gamma \in \mathbb{R}^d$ (learnable scale parameter, initialised to 1)
- $\beta \in \mathbb{R}^d$ (learnable shift parameter, initialised to 0)
- ϵ (small constant, e.g. 10^{-5}, to avoid division by zero)

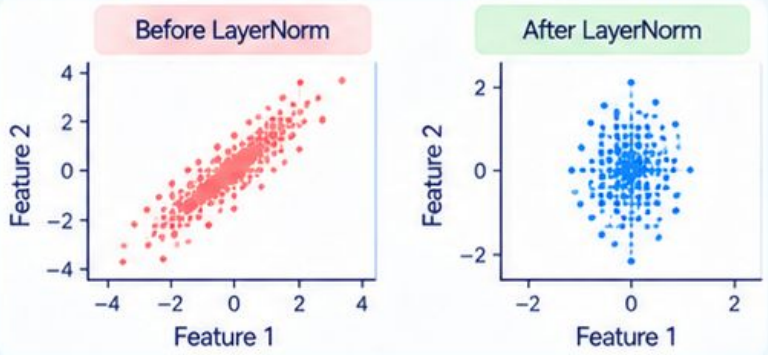
Example: normalising a token representation

Feature (i)	x_i	Normalised value (y_i)
1	2.0	0.96
2	0.5	-1.01
3	1.0	-0.40
4	1.5	0.45
Mean	1.25	0.00
Std (σ)	0.56	1.00

Steps:

1. Compute mean μ of the features.
2. Compute variance σ^2.
3. Normalise: $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$.
4. Apply learnable scale γ and shift β.

Effect on activations



Key benefits

- Improves training stability and convergence.
- Reduces sensitivity to initialisation and learning rate.
- Works well with deep architectures and residual connections.
- Widely used in Transformers and many modern deep learning models.

SIS 3204Y(5)
Natural Language Processing

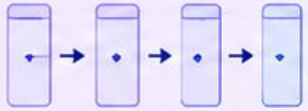
Week 7
Transformers, BERT and Beyond

Transformers, BERT and Beyond

Transformers have revolutionised NLP by using self-attention, enabling models like the BERT to understand context deeply and achieve state-of-the-art performance in many language tasks.

From RNNs to Transformers

RNN / LSTM / GRU



- Sequential processing
- Hard to capture long-range dependencies
- Slower training

Attention Mechanism



- Focus on relevant words
- Better context understanding
- More efficient

Transformer



- Parallel processing
- Multi-head attention
- State-of-the-art performance

What is BERT?

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained model that uses a transformer encoder to learn contextual word representations.

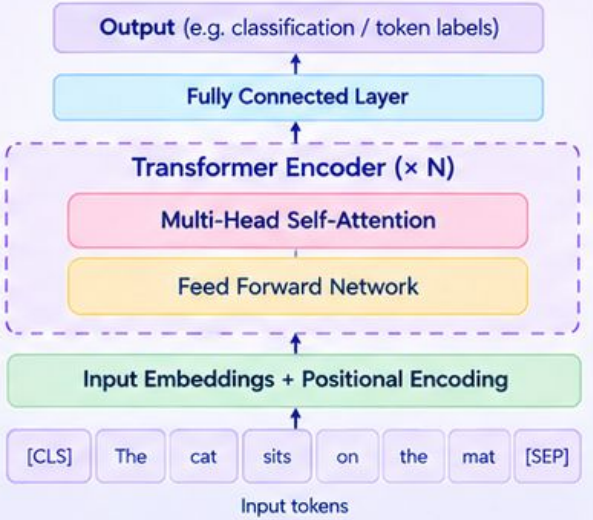
Bidirectional
Uses both left and right context.

Pre-trained
Learned from large text corpora (e.g. Wikipedia, BooksCorpus).

Contextual embeddings
Each word gets a context-aware vector.

Fine-tuning
Adapted for specific downstream tasks (e.g. classification, QA, NER).

BERT Architecture (simplified)



Key Features of BERT

Deep contextual understanding
Considers the full sentence (not just previous tokens).

Pre-trained on massive data
Learns general language knowledge.

Uses special tokens
[CLS] for classification, [SEP] for sentence pairs.

Fine-tuned for many tasks
e.g. sentiment analysis, question answering, named entity recognition.

BERT in Action – Example

Sentence: "The cat sits on the mat."

[CLS]	The	cat	sits	on	the	mat	[SEP]

Contextual embeddings (simplified)

The	[0.12	-0.34	0.56	...]
cat	[0.03	0.41	-0.22	...]
sits	[-0.28	0.19	0.33	...]
mat	[0.21	-0.07	0.45	...]

Each word's representation is influenced by the entire sentence context.

Why It Matters

- Better understanding of meaning and context.
- Improved performance on complex NLP tasks.
- Foundation for modern models like GPT, T5, RoBERTa.
- Enables applications such as chatbots, search, translation and more.

Applications

- Text classification (e.g. sentiment analysis)
- Question answering (e.g. SQuAD)
- Named entity recognition (e.g. people, places, organisations)
- Machine translation (e.g. multilingual models)
- Conversational agents (e.g. chatbots)

SIS 3204Y(5)
Natural Language Processing

Week 7
Transformer Architecture, BERT and GPT

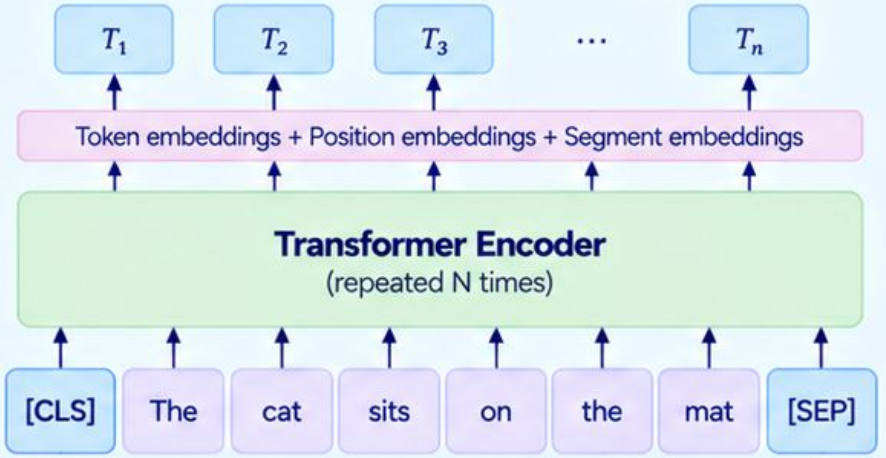
BERT: Bidirectional Encoder Representations from Transformers

BERT is an encoder-only Transformer model pretrained with masked language modelling to learn rich, bidirectional contextual representations.

Key characteristics

- Encoder-only Transformer architecture.
- Uses bidirectional context (information from both left and right).
- Pretrained using large amounts of unlabelled text.
- Produces contextual embeddings for each token.
- Fine-tuned or used as a feature extractor for many NLP tasks.

BERT architecture (encoder-only)



Pretraining objectives

1. **Masked Language Modelling (MLM)**
   Randomly mask some tokens and predict them using both left and right context.

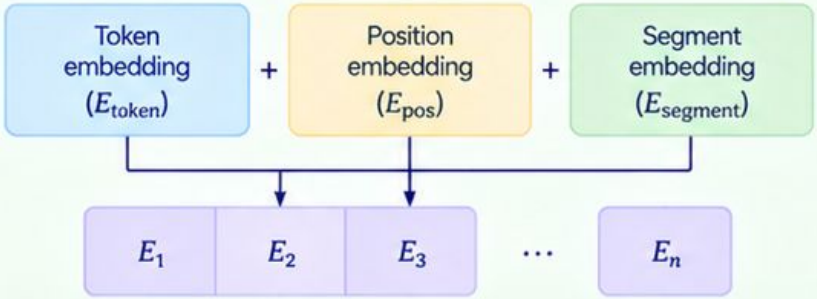
   Input: The cat sits on the [MASK].
   Target: mat

2. **Next Sentence Prediction (NSP)**
   Classify whether two sentences are consecutive in the original text.

   Sentence A: The cat sits on the mat.
   Sentence B: It is sleeping now.
   Target: IsNext (or NotNext)

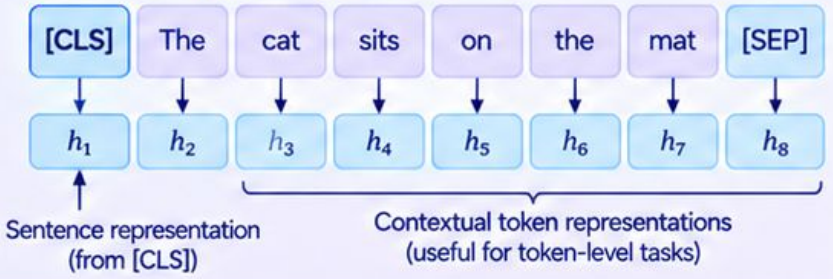
Input representation

Each input token is represented as the sum of three embeddings:



BERT output representations

BERT produces a contextual embedding for each token. The [CLS] token representation is often used for sentence-level tasks (e.g. classification).



Applications of BERT

- Text classification (e.g. sentiment analysis)
- Named entity recognition (NER)
- Question answering
- Text similarity and semantic search
- Text summarisation (with modifications)
- Information retrieval
- Feature extractor for downstream models (e.g. in RAG systems)

SIS 3204Y(5)
Natural Language Processing

Week 7
Transformer Architecture, BERT and GPT

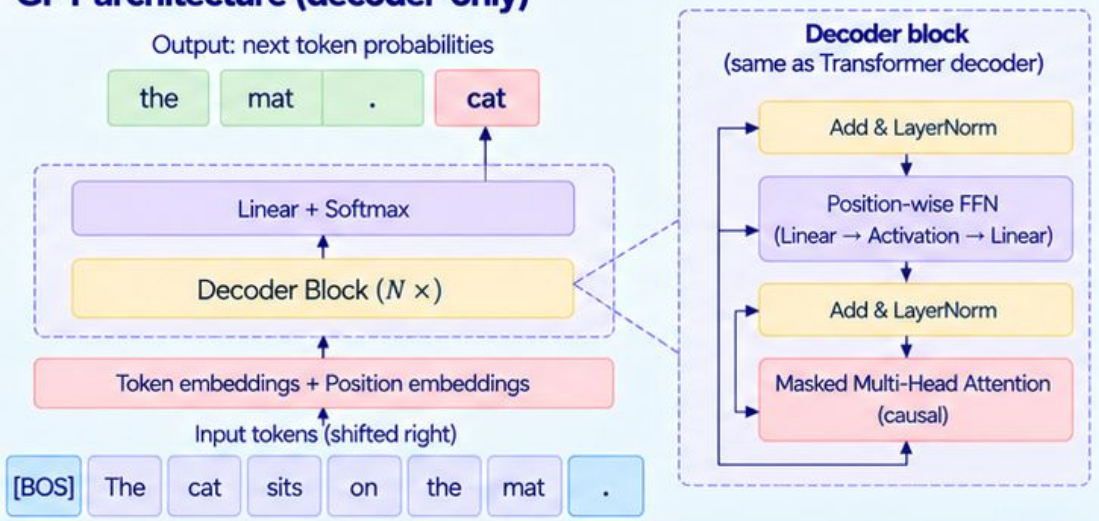
GPT: Decoder-Only Transformer for Language Generation

GPT (Generative Pre-trained Transformer) is a decoder-only Transformer model trained to predict the next token in a sequence.

Key characteristics

- Decoder-only Transformer architecture.
- Uses causal (masked) self-attention to only attend to previous tokens.
- Trained with next-token prediction objective.
- Pretrained on large amounts of unlabelled text.
- Excellent at generating human-like text, following instructions and reasoning (in larger models).

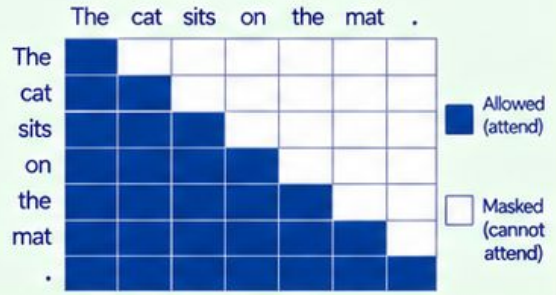
GPT architecture (decoder-only)



Causal (masked) self-attention

- Each position can only attend to previous positions (not future tokens).
- A causalmask is applied to prevent attending to future tokens.

Attention mask (causal)



Training objective: next-token prediction

Given a sequence of tokens, predict the next token at each position.

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1}) = \text{softmax}(h_t W)$$

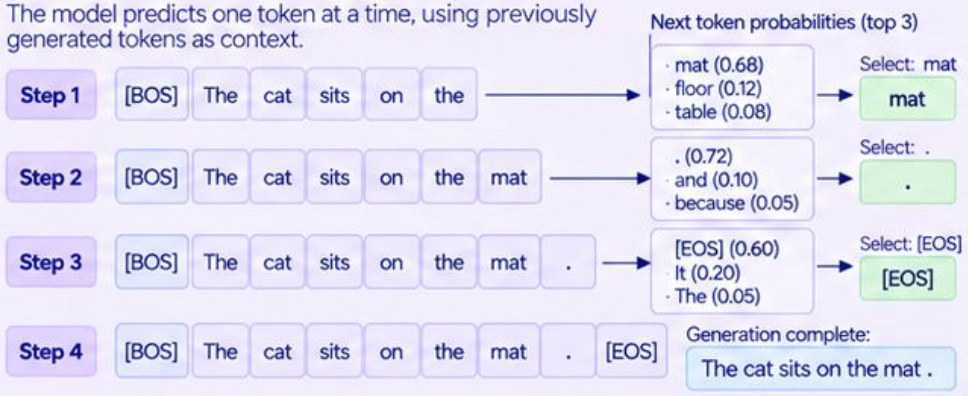
Training data (example):

Input:	The cat sits on the mat .
Targets:	cat sits on the mat . [EOS]

- Trained using large-scale self-supervised learning on internet text.
- Learns grammar, factual knowledge and reasoning capabilities.
- No labelled data is required.

Text generation example

The model predicts one token at a time, using previously generated tokens as context.



Applications of GPT (LLMs)

- Text generation (content, dialogue, summarisation).
- Conversational agents (e.g. ChatGPT).
- Code generation and assistance.
- Translation and language adaptation.
- Question answering.
- Reasoning (in larger models).
- Creative writing and ideation.

Examples of decoder-only LLMs
GPT (OpenAI), GPT-3, GPT-4, GPT-4o, Llama (decoder-only), Mistral.

SIS 3204Y(5)
Natural Language Processing

Week 7
Transformer Architecture, BERT and GPT

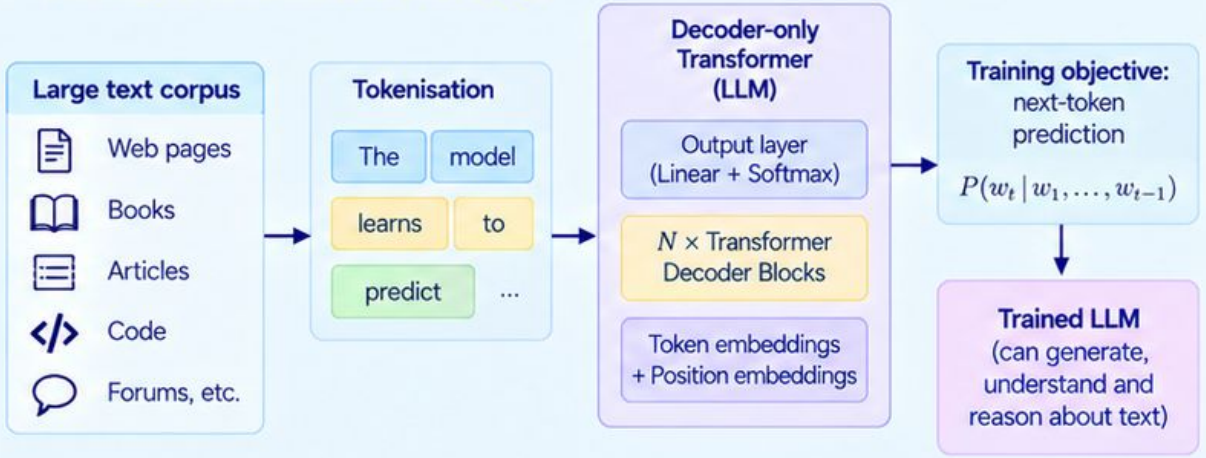
Large Language Models (LLMs)

Large Language Models (LLMs) are large-scale language models trained on massive amounts of text to understand and generate human-like text.

What are LLMs?

- LLMs are deep neural networks (typically decoder-based Transformers) with billions of parameters.
- Trained on massive and diverse text data from the internet, books, code, etc.
- Learn to predict the next token in a sequence (self-supervised learning).
- Can perform a wide range of tasks, from text generation to reasoning, with minimal task-specific training.

Training of LLMs (overview)

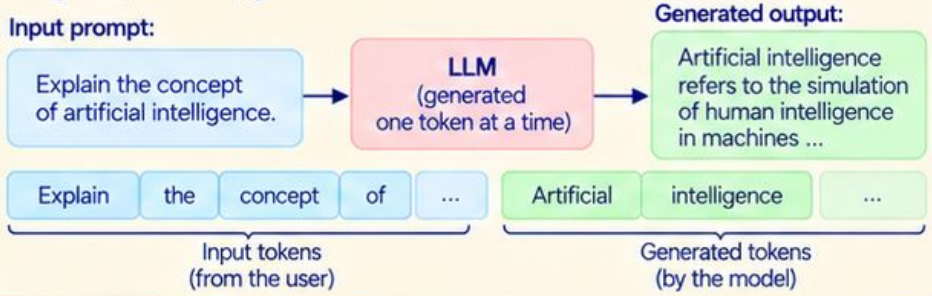


Capabilities of LLMs

- Generate coherent and contextually relevant text.
- Answer questions and provide explanations.
- Perform summarisation, translation, text classification, and more.
- Follow instructions (instruction tuning).
- Can use additional tools, data or retrieval (e.g. RAG) to access up-to-date or domain-specific information.

Inference (text generation)

At inference time, the model generates tokens one at a time, using the previously generated tokens as context.



Examples of LLMs

Model	Organisation	Parameters (approx.)	Notes
GPT-3	OpenAI	175B	Early large-scale LLM
GPT-3.5	OpenAI	~175B	Improved training and alignment
GPT-4 / GPT-4o	OpenAI	Unknown (very large)	Multimodal (text, image, etc.)
Llama 3	Meta	8B / 70B	Open weights
Gemini	Google	Various (scaled)	Multimodal
Mistral	Mistral AI	7B / 8×7B	Efficient models

Key points

- Trained on very large and diverse datasets using next-token prediction.
- Have strong language understanding and generation capabilities.
- Can be adapted through instruction tuning, fine-tuning or prompting.
- Often used with retrieval (RAG) and tools to improve accuracy and up-to-date information.
- Pose challenges related to bias, misinformation, safety and ethical use.

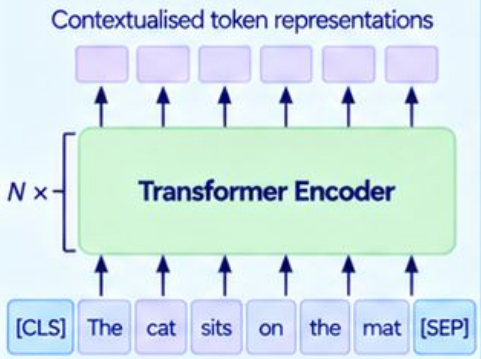
SIS 3204Y(5)
Natural Language Processing

Week 7
Transformer Architecture, BERT and GPT

Encoder-Only vs Decoder-Only vs Encoder–Decoder Transformers

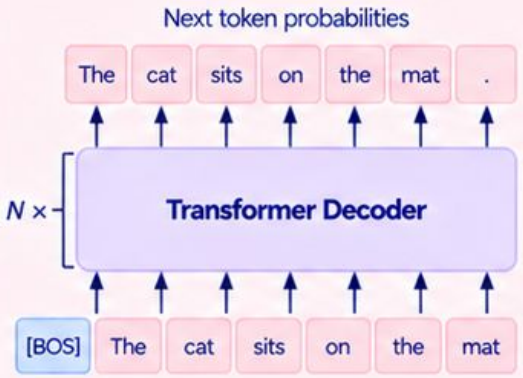
Transformer models can be configured in different ways depending on the task: encoder-only, decoder-only or encoder–decoder.

Encoder-only Transformer
(e.g. BERT, RoBERTa)



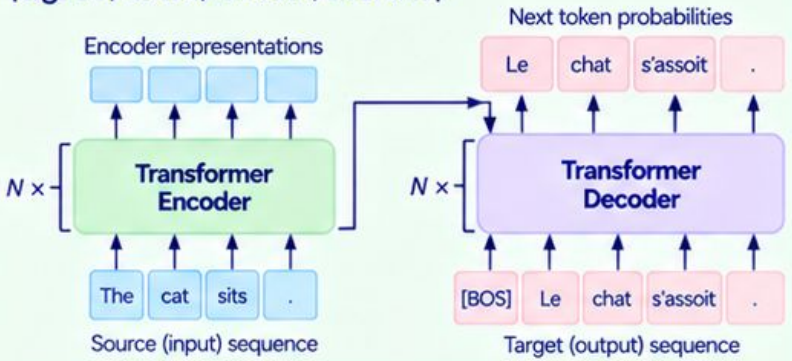
- Processes the entire input sequence using bidirectional self-attention.
- Produces contextual representations for each token.
- Trained mainly using masked language modelling (MLM).
- Suitable for language understanding tasks.

Decoder-only Transformer
(e.g. GPT, Llama, Mistral)



- Uses causal (masked) self-attention (attends only to previous tokens).
- Generates the next token autoregressively.
- Trained using next-token prediction.
- Excels at text generation and open-ended tasks.

Encoder–Decoder Transformer
(e.g. T5, BART, Marian, mBART)



- Uses an encoder to understand the source input and a decoder to generate the target output.
- Decoder uses both causal self-attention and cross-attention to the encoder outputs.
- Trained for sequence-to-sequence tasks (e.g. translation, summarisation).
- Well suited for tasks where the input and output are different sequences.

Comparison of model types

Aspect	Encoder-only (BERT)	Decoder-only (GPT)	Encoder–Decoder (T5/BART)
Self-attention	Bidirectional (attends to all tokens)	Causal (attends to previous tokens only)	Encoder: bidirectional Decoder: causal + cross-attention
Training objective	Masked Language Modelling (MLM)	Next-Token Prediction	Sequence-to-Sequence (e.g. denoising, translation)
Input / output	Single sequence (input)	Single sequence (input → output)	Input sequence → output sequence
Main strength	Language understanding, contextual representations	Text generation, open-ended tasks	Sequence transformation (e.g. translation, summarisation)
Typical tasks	Text classification, NER, QA, semantic search	Text generation, chatbots, creative writing	Machine translation, summarisation, paraphrasing, text-to-text tasks
Examples	BERT, RoBERTa, ALBERT, DeBERTa	GPT, Llama, Mistral, GPT-4	T5, BART, Marian, mBART

Key takeaways

- The three Transformer variants share the same core architecture but differ in how they use attention and in their training objectives.
- Encoder-only models (BERT) are used for understanding text.
- Decoder-only models (GPT) are used for generating text.
- Encoder–decoder models are used for tasks where the input and output are different sequences.
- Choosing the right model depends on the task and the type of input-output you need.

How BERT and GPT Process the Same Sentence

The same input text is processed differently by BERT (bidirectional) and GPT (autoregressive).

Example sentence

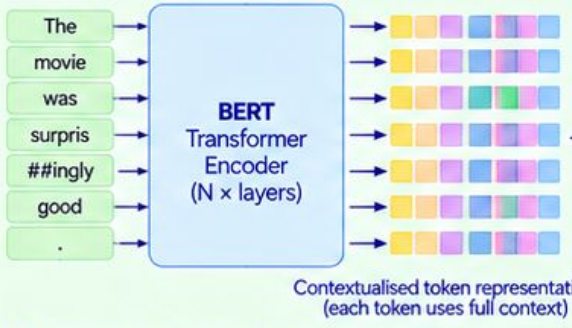
The movie was surprisingly good .

Tokenisation (example using WordPiece/BPE)

The	movie	was	surpris	##ingly	good	.
1	2	3	4	5	6	7

BERT — Bidirectional understanding (encoder-only)

Each token attends to all other tokens in the sentence (both left and right).



Contextualised token representations
(each token uses full context)

Attention for "good"

The	0.05
movie	0.10
was	0.15
surpris	0.20
##ingly	0.25
good	0.20
.	0.05

Downstream task: Sentiment classification

"The movie was surprisingly good."

↓

Classification layer

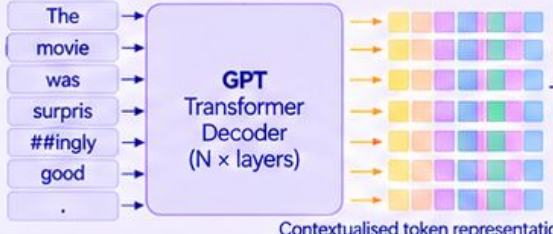
↓

Positive

- Uses bidirectional self-attention (no causal mask).
- Produces a contextual representation for each token.
- Trained with masked language modelling (predicts masked tokens).
- Well suited for understanding tasks (classification, NER, question answering).

GPT — Autorgressive generation (decoder-only)

Each token attends only to previous tokens (left context) due to causal masking.



Contextualised token representations
(each token uses only previous tokens)

Causal attention for "good"

The	0.05
movie	0.10
was	0.15
surpris	0.25
##ingly	0.30
good	-
.	-

Cannot attend to future tokens

Next-token prediction

Input: The movie was surprisingly

↓

GPT

↓

Predicted next token:

good

Continue generation:

.

- Uses causal (masked) self-attention (cannot see future tokens).
- Trained to predict the next token in the sequence.
- Generates text one token at a time (autoregressive).
- Well suited for open-ended text generation and conversational applications.

SIS 3204Y(5)
Natural Language Processing

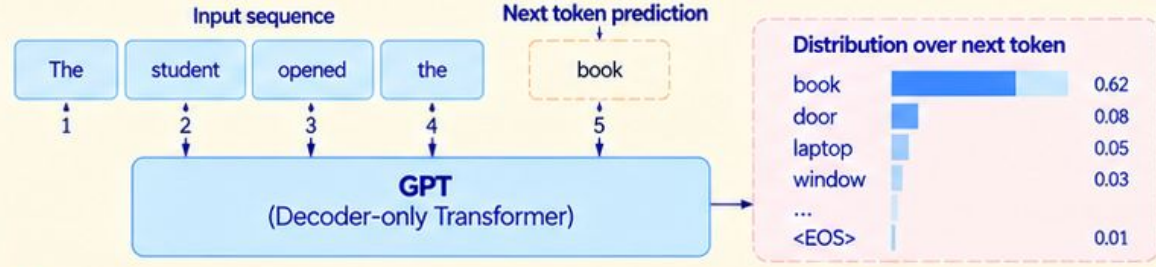
Week 7
Transformer Architecture, BERT and GPT

From GPT to Instruction-Tuned LLMs

Large language models (LLMs) build on the GPT architecture and are further trained to follow instructions and perform a wide range of tasks.

1 Base GPT model — Next-token prediction

Learns to predict the next token from large amounts of text using causal self-attention.



2 Instruction-tuned LLM — Following user instructions

Builds on a GPT-like model and is further trained using techniques such as:

- Supervised fine-tuning (instruction–response pairs)
- Reinforcement learning from human feedback (RLHF)
- Alignment to be helpful, safe and follow instructions

User prompt (instruction)

Summarise the following customer complaint in one sentence:

"The delivery was late, the item was damaged and customer service did not respond."

→

Instruction-tuned LLM
(GPT-based architecture)

- Understands the instruction
- Uses large-scale knowledge
- Generates a helpful response
- Follows safety and alignment rules

→

Model response

"The customer experienced a delayed delivery, received a damaged item and did not receive a response from customer service."

3 Key differences

Aspect	Base GPT model	Instruction-tuned LLM
Main objective	Next-token prediction	Follow instructions and be helpful
Training data	Large amounts of general text	General text + instruction–response pairs + human feedback
Behaviour	Completes text based on context	Understands and follows user instructions
Typical uses	Text generation, language modeling	Chatbots, summarisation, Q&A, coding assistance, document analysis and more

Key takeaways

- GPT is a decoder-only model trained for next-token prediction.
- Instruction-tuned LLMs use the same underlying architectcre but are further trained to follow instructions and generate helpful, aligned responses.
- LLMs can perform a wide range of tasks using natural language prompts (zero-shot or few-shot).

SIS 3204Y(5)
Natural Language Processing

Week 7
Transformer Architecture, BERT and GPT

Training vs Inference in Transformers

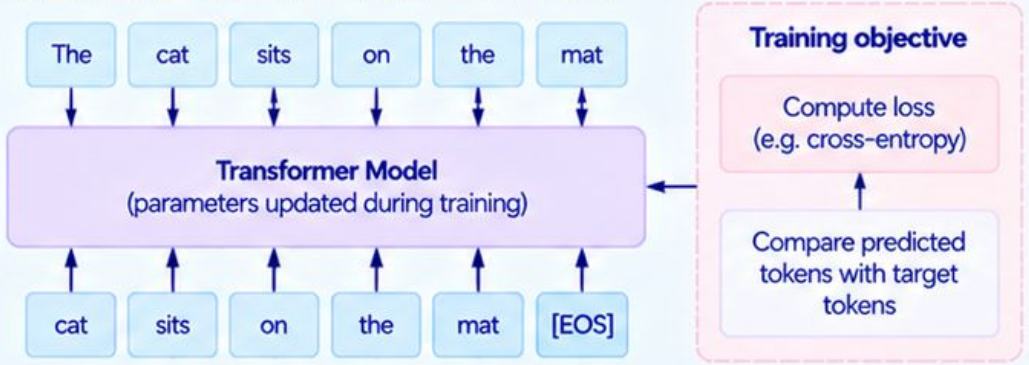
Transformers are first trained on large amounts of text data and then used for inference to make predictions on new inputs.

Key concepts

- **Training:** The model learns from data by adjusting its parameters to minimise a loss function.
- **Inference:** The trained model is used to make predictions on new inputs.
- Training and inference differ in the input format, use of target tokens, and whether the model updates its parameters.
- The training objective depends on the model type (e.g. MLM for BERT, next-token prediction for GPT).

Training process (example: next-token prediction)

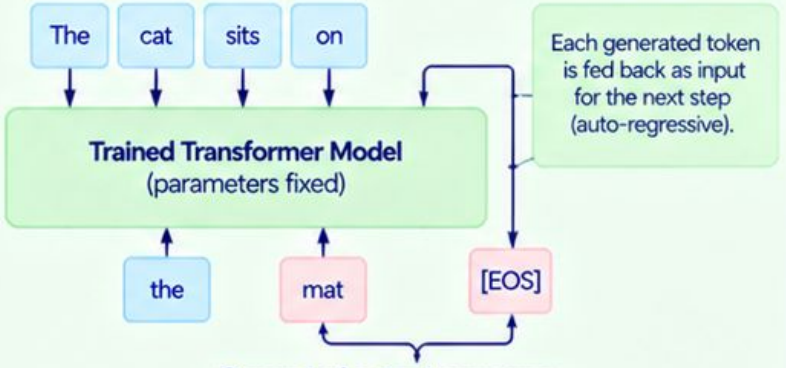
The model sees the input sequence with the next token shifted to the right during training, and learns to predict the next token.



Target sequence (shifted right)
Model learns to predict the next token at each position.

Inference process (generation)

The trained model generates the next token one at a time, using its own previous predictions.

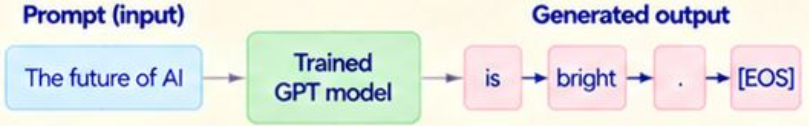


Generated output sequence
Model uses learned probabilities to generate tokens one at a time.

Training vs inference comparison

Aspect	Training	Inference
Purpose	Learn model parameters	Make predictions using trained model
Input	Input sequence and target tokens	Only input sequence (or prompt)
Output	Predicted tokens for all positions	Generated tokens one at a time
Parameters	Updated using backpropagation	Fixed (no updates)
Objective	Minimise loss (e.g. cross-entropy)	Generate most likely next-token)
Computation	Parallel (all tokens processed together)	Sequential (one token at a time)
Special tokens	Uses [CLS], [SEP], [BOS], [EOS] as needed	Uses prompt and may stop at [EOS]

Example: GPT text generation



- During inference, the model generates one token at a time based on the probability distribution over the vocabulary.
- Generation stops when a stopping criterion is met (e.g. [EOS] token or maximum length).
- Different decoding strategies (greedy, beam search, nucleus sampling) can be used to control the output.

Key takeaways

- Training and inference have different input formats and objectives.
- During training, the model learns to predict tokens and its parameters are updated.
- During inference, the trained model generates tokens one at a time without updating its parameters.
- The process is similar for BERT and GPT, but with different training objectives (MLM vs next-token prediction).

SIS 3204Y(5)
Natural Language Processing

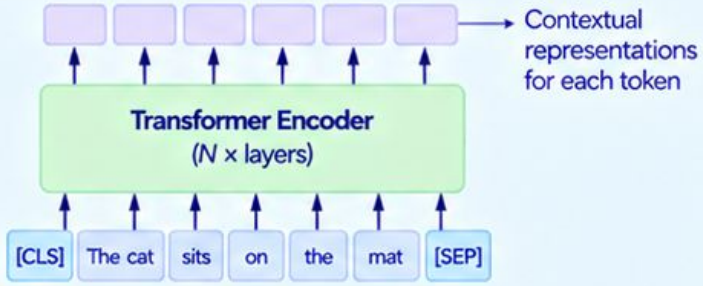
Week 7
Transformer Architecture, BERT and GPT

Week 7 – Final Recap

Transformers can be used in three main configurations: encoder-only, decoder-only or encoder–decoder, depending on the task.

Encoder-only (e.g. BERT)

Bidirectional context for language understanding.

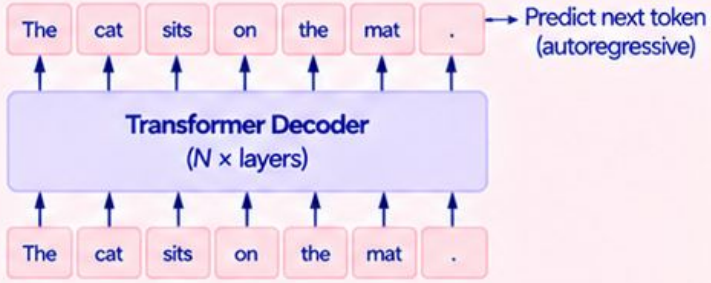


Key characteristics

- Uses bidirectional self-attention (attends to all tokens).
- Trained using masked language modelling (MLM).
- Excels at language understanding tasks.
- **Examples:** BERT, RoBERTa, ALBERT, DeBERTa.

Decoder-only (e.g. GPT)

Autoregressive generation for open-ended text.

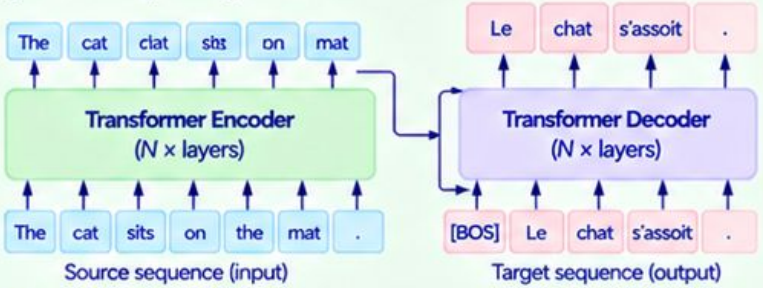


Key characteristics

- Uses causal (masked) self-attention (attends only to previous tokens).
- Trained using next-token prediction.
- Generates text one token at a time.
- Excels at text generation and open-ended tasks.
- **Examples:** GPT, Llama, Mistral, Claude.

Encoder–Decoder (e.g. T5, BART)

Sequence-to-sequence modelling for tasks with different input and output sequences.



Key characteristics

- Uses an encoder to understand the input and a decoder to generate the output.
- Uses cross-attention in the decoder.
- Well suited for sequence-to-sequence tasks.
- **Examples:** T5, BART, Marian, mBART.

When to use which model?

Task type	Recommended model	Examples
Text classification	BERT (encoder-only)	Sentiment analysis, spam detection
Named entity recognition	BERT (encoder-only)	Extracting people, organisations, dates
Question answering	BERT (encoder-only)	Reading comprehension, document QA
Text generation	GPT (decoder-only)	Chatbots, content creation
Summarisation	GPT or Encoder–Decoder	Article summarisation, report generation
Machine translation	Encoder–Decoder	English → French, multilingual translation
Dialogue systems	GPT (decoder-only)	Conversational agents
Paraphrasing / text-to-text	Encoder–Decoder (e.g. T5)	Rewriting, style transfer

Key takeaways

1. The Transformer architecture can be configured as encoder-only, decoder-only or encoder–decoder.
2. BERT uses masked language modelling and is trained for understanding.
3. GPT uses next-token prediction and is trained for generation.
4. Encoder–decoder models are used for tasks where the input and output are different sequences.
5. The choice of model depends on the task, the type of input and output you need, and the desired behaviour (understanding vs generation).

Next steps

Practical exploration
Try out pre-trained models (e.g. BERT, GPT, T5) on real NLP tasks.

Experiment with fine-tuning
Apply transfer learning to adapt models for specific datasets.

Compare models
Evaluate performance on similar tasks (e.g. classification, summarisation, translation).

Consider responsible AI
Be aware of model limitations, biases and ethical considerations when using LLMs.